This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Self-Attention Gated Linear Recurrent Units for Sequential Recommendation

Jian Yang, Tengfei Bi, Jiajin Huang, and Mi Li, *Member, IEEE*

***Abstract*—Sequential recommendation is of utmost importance for personalized suggestions across various domains, and its effectiveness hinges critically on accurately capturing the intricate sequential dependencies among items. Two advanced architectures are widely adopted for this purpose: self-attention mechanisms (SAM), which excel at modelling global dependencies in sequences, and linear recurrent units (LRU), which specialize in capturing local sequential patterns. However, SAM inherently lacks the capability to convey sequential order information, while LRU may overemphasize item ordering. Crucially, while the complementary strengths of SAM and LRU architectures are promising, their direct integration risks overfitting in noisy real-world recommendation scenarios, as overly refined representations may predominantly capture training noise rather than generalizable patterns. To tackle this issue while preserving the advantages of both architectures, we design a dual-pronged solution: perturbation injection during recurrence enhances robustness, while Gibbs distribution-based uncertainty scaling reduces noise sensitivity. Integrating these strategies, we propose a self-attention gated linear recurrent units for sequential recommendation (SLSRec) model. Specifically, through a novel self-attention gated unit, SLSRec synergistically integrates SAM's global modeling capability with LRU's sequential processing strength. The model is optimized using a comprehensive objective function that combines cross-entropy recommendation loss, perturbation injection loss, and uncertainty-incorporated recommendation loss. Extensive experiments on real-world datasets demonstrate that SLSRec outperforms state-of-the-art methods, showing particular effectiveness in handling noisy interactions and recommending long-tail items.**

***Index Terms*—Linear recurrent unit (LRU), perturbation injection, self-attention mechanism (SAM), sequential recommendation, uncertainty scaling.**

Received 7 January 2025; revised 12 May 2025, 1 November 2025, 16 January 2026, and 24 February 2026; accepted 24 March 2026. This work was supported by the National Natural Science Foundation of China under Grant 62576245, Grant 62573008, Grant 61602017, Grant 61632014, and Grant 61627808. *(Corresponding author: Mi Li.)*

The authors are with the School of Information Science and Technology, Beijing University of Technology, Beijing 100124, China, and also with Beijing International Collaboration Base on Brain Informatics and Wisdom Services, Beijing 100124, China (e-mail: jianyang@bjut.edu.cn; bitengfei@emails.bjut.edu.cn; jhuang@bjut.edu.cn; limi@bjut.edu.cn).

Digital Object Identifier 10.1109/TCSS.2026.3679563

I. Introduction

RECOMMENDER systems occupy a pivotal position in various internet-based application domains, notably e-commerce platforms [1], social networks [2], [3], video streaming services [4] and music applications [5], by tailoring personalized suggestions to users' preferences. Among these sophisticated systems, sequential recommendation stands out as a cornerstone, attracting considerable research attention for its ability in predicting users' subsequent actions based on their historical behavioral sequences. The evolution of sequential recommendation has been marked by a transition from classical Markov chain methods [6], [7] to advanced deep learning architectures, in particular recurrent neural networks (RNN) [8], [9], [10] and convolutional neural networks (CNN) [11], [12], which have revolutionized the landscape of sequence modelling.

In recent years, the Transformer architecture has revitalized sequential recommendation and has become one of the mainstream techniques for accurately delivering personalized and high-quality recommendations to users. Under the architecture, self-attention mechanism (SAM)-based models [13], [14] skillfully construct a self-attention matrix by deeply mining the inherent rules of transaction sequence data, which not only accurately captures the complex dependencies and relative importance among items in the sequence, but also promotes a significant improvement in recommendation accuracy. Building upon this line of research, newer approaches like ScaleRec [15] further refine the capability of SAM to capture patterns at different granularities. By extending the architecture of the Transformer, linear recurrent units (LRU)-based models [16], [17] have shown remarkable potential and broad prospects in sequential recommendation by finely analyzing the sequential logic of items in the transaction sequence and revealing their dynamic dependency relationships. More recently, architectures like SIGMA [18] have explored selective gating mechanisms within state-space models, demonstrating another promising direction for efficient and effective sequential modeling. Although SAM-based models and LRU-based models have demonstrated excellent performance in the area of sequential recommendation, they both face a common challenge: *How to appropriately capture the order relationship of items in users' historical transaction sequences?*

In sequential recommendation, some order relationships among items in a sequence are very important, while others are relatively unimportant [19]. For an illustrative example, consider user Janet's purchase sequence: {iPhone, earphones,

2329-924X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

phone charger, phone data cable, Harry Potter and the Philosopher's Stone, Harry Potter and the Chamber of Secrets}. Among these items, four are related to mobile phones, and two are the first two Harry Potter books. There is no strict order among the four phone related items, and it does not matter whether a user buys earphones, a data cable or charger first after purchasing a mobile phone. However, the order in which the books in the Harry Potter series are read is crucial to understanding the story, and the order among them is very important.

SAM-based models adopt a global perspective to assign weights to previous items, often prioritizing those related to the mobile phone while de-emphasizing Harry Potter related items and ignoring the order among Harry Potter related purchases [20]. Conversely, LRU-based models utilize a local manner to emphasize the influence of the immediately preceding item on the subsequent one, enabling them to accurately sequence four items related to mobile phones. However, this frequently leads to recommendations that are narrowly confined to earphones after an iPhone purchase, neglecting chargers and data cables. Ultimately, due to their inherent architectural constraints, neither SAM nor LRU can effectively strike a balance in capturing item order information.

A direct and effective approach to enhancing sequential recommendation systems lies in combining SAM with LRU. Recent efforts have explored similar synergistic designs; for instance, BiSGR-Att [21] integrates bidirectional gated recurrent networks with linear attention to capture sequential dependencies from dual perspectives. While this integration leverages SAM's ability to capture intricate cross-item patterns alongside LRU's proficiency in modeling local temporal transitions, it introduces an inherent risk of amplifying model complexity. Other contemporary approaches address related challenges, such as enhancing explainability through disentangled representations [22] or separating long-term and short-term user interests for debiasing [23]. These works highlight the multifaceted nature of improving sequential recommenders beyond mere accuracy. When these mechanisms are merged without careful calibration, the resulting model may generate overly refined representations that overfit both meaningful patterns and training data noise, ultimately degrading generalization performance. This challenge is particularly pronounced in real-world recommendation systems that rely on implicit feedback, where noisy interactions and mislabeled items are prevalent, as highlighted in [24]. Exploring alternative paradigms, such as frequency-domain filtering [25], [26], also represents an ongoing effort to build more robust sequence representations.

The overfitting phenomenon arises from two interrelated factors. First, the quadratic computational complexity of SAM and the linear parameter growth of LRU jointly create a high-capacity model that is prone to memorizing spurious correlations in limited training data. Second, implicit feedback often contains random behaviors or misclicks; SAM's global attention may inadvertently assign excessive weight to these noisy interactions, while LRU's sequential updates could propagate such errors through hidden states, compounding their impact. These conflicting optimization goals result in unstable parameter updates, further exacerbating overfitting. Empirical evidence from Section IV-C (Fig. 3) demonstrates that direct fusion of SAM and LRU significantly amplifies overfitting to training data, including noise, thereby reducing performance on validation sets.

To effectively mitigate the risk of overfitting associated with the fusion of SAM and LRU, we adopt a dual-pronged strategy. On one hand, we enhance the robustness of the model by perturbing its recurrent process. This is achieved by randomly substituting an item representation in each layer with an embedding drawn from a uniform distribution. Such random perturbation is intended to mitigate overfitting resulting from noisy data, thereby improving the model's robustness. On the other hand, we augment the output flexibility by incorporating uncertainty scaling factors into the prediction outcomes. Uncertainty scaling, which is grounded in a Gibbs distribution, serves as a probabilistic regularization technique to alleviate the impact of inherent noise on the predicted values.

Based on this idea, we propose a self-attention gated linear recurrent units for sequential recommendation (SLSRec) model, which combines SAM and LRU into parallel branches through gating mechanisms, thereby complementing their respective advantages. SLSRec inherits the parallel efficiency of SAM, which concurrently assigns attention weights to all items, and the sequential capability of LRU, which systematically traverses sequences step-by-step. Furthermore, SLSRec mitigates overfitting caused by the fusion of SAM and LRU through a dual-pronged strategy: perturbation injection and uncertainty scaling techniques. By executing the LRU and SAM components in parallel and integrating them via elementwise multiplication and augmented with the anti-overfitting strategy, the proposed SLSRec model effectively captures intricate sequential dependencies while ensuring robustness against overfitting.

In summary, the first challenge addressed in this article is how to appropriately capture the order relationship of items in users' historical interaction sequences. Our solution is to integrate SAM and LRU through a gating mechanism (see the model architecture of SLSRec in Fig. 1). Overall performance in Section IV-B and ablation experiments in Section IV-C validates that this integration outperforms using either model alone. The second challenge we confront is how to mitigate the overfitting risk caused by noisy interactions during model integration. Our strategy is to employ perturbation injection and uncertainty scaling during model training (see the model training diagram of SLSRec in Fig. 2). Both ablation experiments in Section IV-C and noise and long-tail scenarios experiments in Section IV-D demonstrate the effectiveness of these techniques.

The main contributions of this article are as follows.

1) In terms of model architecture, we propose to integrate SAM and LRU to balance item order considerations. Using SAM as a gating signal and combining it into the iterative LRU process, we enable the LRU to fully capture sequential information.
2) In terms of model training, we propose to combine perturbation injection and aleatoric uncertainty strategies to improve recommendation performance. We develop sophisticated uncertainty scaling to assess uncertainty from both sequence and target item representations.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

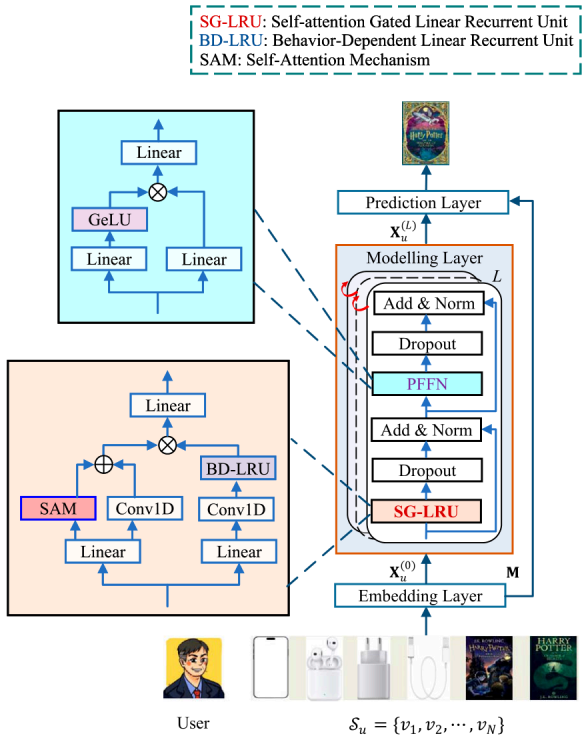


Fig. 1. Model architecture overview of the proposed SLSRec model, consisting of an embedding layer, a modelling layer and a prediction layer.

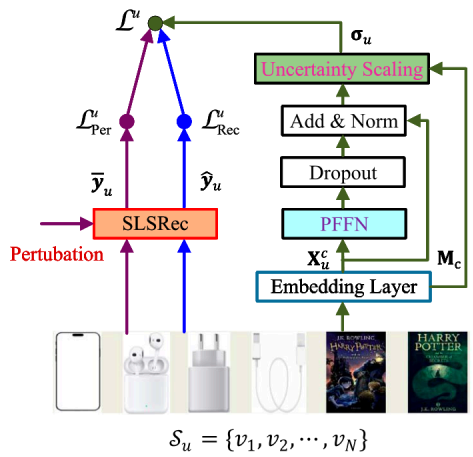


Fig. 2. Model training of the proposed SLSRec, calculating three losses through different channels (denoted by ●): a recommendation loss (blue), a perturbation injection loss (purple) and an uncertainty incorporated recommendation loss (green).

3) Extensive experimental results demonstrate that the proposed model improves the recommendation performance, even for noisy interactions and long-tail items.

II. Related Work

A. Sequential Recommendation

In the field of sequential recommendation, a series of innovative advances have dramatically changed the landscape of personalized recommendations. Early efforts focus on exploiting the synergy between Markov chain [6] and matrix factorization techniques [7], meticulously building models that capture user preferences rooted in their historical interaction patterns. MF [27], FISM [28] and FPMC [7] are typical representatives of matrix factorization techniques in the field of recommendation systems. FISM combines factorization techniques, while FPMC integrates Markov chain methods. Together, these methods explore innovative technological frameworks that lay the groundwork for subsequent advances in sequential recommendation. The advent of deep learning methods [29] represents a paradigm shift, ushering in a new era of increased expressiveness and capability for sequential recommendation. Pioneering the fusion of neural networks with sequential recommendation, CNN [19], [30], RNN [8], [9], [10] and Transformers [31], open up new avenues for unravelling the complexity of sequential data. For example, GRU4Rec [8], [9] and NARM [32] are recommendation algorithms carefully designed on the RNN architecture, which are early attempts to fuse deep learning paradigms with the frameworks of recommender systems.

Transformer-based architectures [13], [33] leverage their sophisticated SAM to intricately identify and exploit relationships among items, resulting in significant gains in prediction accuracy. SASRec [13], an exemplary model in the field of Transformer-based architectures, achieves remarkable recommendation performance, thereby establishing a benchmark in the field of sequential recommendation. BERT4Rec [33] innovates by incorporating a bidirectional encoding architecture where the historical sequence of user interactions is conceptualized as a form of textual data. ScaleRec [15] captures item correlations at various scales, further enhancing the representational power of attention-based methods. In addition, research shows that multi-layer perceptron (MLP)-based models, in particular FMLP-Rec [25], adeptly balance prediction accuracy with computational efficiency. Exploring new dimensions, WEARec [26] utilizes wavelet-enhanced adaptive frequency filters to learn representations, offering a different perspective on modeling sequential patterns.

More recently, the revitalization of LRU [16], firmly grounded in the robust architecture of state space models [34], marks a significant leap forward. Using innovative techniques of linearization, diagonalization, stable exponential parameterization [35], [36], [37] and tailored initialization strategies, LRU becomes the cornerstone of powerful and efficient state space-based models. LRURec [16] pioneers the integration of LRU into sequential recommendation and sets the benchmark for LRU-driven recommendations. Building on this success, RecBLR [17] extends the concept by integrating real-valued LRU with gating mechanisms, thereby enhancing their adaptability to dynamic user behavior and achieving overall performance improvements. Further advancing this line of work, SIGMA [18] introduces selective gating within the Mamba architecture, demonstrating the potential of combining state-space models with sophisticated gating mechanisms for sequential recommendation.

SAM-based models inherently do not capture the temporal properties of items in the sequence well (although they introduce item positions into the embedding) [20], while LRU-based

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

models overemphasize the order of items. In contrast to these existing models, we consider properly utilizing item order by fusing SAM and an LRU architecture. Our work shares a similar goal with models like BiSGR-Att [21], which also attempts to synergize gated recurrent networks with attention mechanisms. However, our proposed SLSRec distinguishes itself through its specific gating mechanism for parallel branches and, more importantly, its dual-pronged anti-overfitting strategy involving perturbation injection and uncertainty scaling.

Addressing inherent noise in data has always been one of the key concerns in sequential recommendation. For example, FMLP-Rec [25] uses a learnable filter to remove noise from the sequence, and unlike the previous explicit denoising method, STOSA [38] uses a probabilistic embedding layer to represent each item, which indirectly improves the robustness of the model to noise. Relatedly, improving the quality of representations has been tackled from different angles, such as learning disentangled representations for better explainability [22] or separating long- and short-term interests to reduce bias [23]. Although denoising is beyond the scope of this article, the perturbation injection approach we use has a similar idea to STOSA. The goal of perturbation injection is not to identify and remove noise, but to introduce disturbances within the model to reduce over-reliance on the noise inherently present in the interaction data (e.g., as described in [39]), *i.e.*, it is used as a regularization technique to improve the stability of the model.

B. Uncertainty Estimation in Recommender Systems

The notions of epistemic uncertainty and aleatoric uncertainty, originally introduced in computer vision [24], [40], have subsequently been adapted and applied to address real-world challenges [41], [42]. Sequential recommendation also faces uncertainty, and considerable efforts have been made to model the two fundamental types of uncertainty inherent in recommender systems.

One technology roadmap focuses on epistemic uncertainty, where most studies tackle the cold-start problem to generate different recommendations [43], [44]. For example, STOSA [38] employs Gaussian embedding, which incorporates randomness into embeddings, to dynamically capture the uncertain preferences displayed by certain users. Another stream of research focuses on modelling aleatoric uncertainty from a data-centric perspective. For example, AUR [45] focuses on a single task and leverages a Gaussian process-based uncertainty estimator to generate recommendations while accounting for mislabeling issues. In addition, Kendall et al. address multitask scenarios, utilizing uncertainty to assess the importance of different tasks [46]. BDL [47] integrates both epistemic uncertainty and aleatoric uncertainty into a reinforcement learning-based recommendation framework, aiming to optimize the exploration-exploitation balance while incorporating model predictions and uncertainties. In the context of knowledge distillation, UKD [48] introduces aleatoric uncertainty to mitigate the effect of inherent noise in pseudo-labels.

Our work follows the data driven approach, but tackles the inherent noise challenge specific to implicit sequential recommendations. We model the prediction value as a Gibbs distribution, which provides a novel perspective to address this issue in sequential recommendation.

III. Methodology

Previous studies have shown that SAM and LRU can effectively capture sequential dependencies, respectively. However, they have inherent limitations in either ignoring or overemphasizing the importance of item order information in a sequence, which limits their recommendation performance. In this section, we present how to integrate SAM and LRU to adaptively utilize the item order in a sequence, and how to improve the robustness and long-tail effectiveness of the model by using perturbation injection and aleatoric uncertainty techniques.

A. Problem Description

In sequential recommendation, let $\mathcal{U}$ denote the user set, $\mathcal{I}$ denote the item set, and $\mathcal{S}_u = \{v_1, v_2, \cdots, v_N | v_t \in \mathcal{I}, 1 \leq t \leq N\}$ be the chronologically ordered interaction sequence for user $u \in \mathcal{U}$, where N is the maximum sequence length. Given the interaction sequence $\mathcal{S}_u$, the task of sequential recommendation is to predict the next item to interact with user u. Typically, the task is transformed into computing the probabilities of user u interacting with all candidate items and recommending the top-K items with the highest probabilities to the user.

B. Model Architecture

The overall architectural design of our proposed SLSRec model is illustrated in Fig. 1. It comprises an embedding layer, a modelling layer and a prediction layer. The modelling layer is its core part and consists of stacked L recurrent layers each of which has two modules: a self-attention gated linear recurrent unit (SG-LRU) and a position-wise feed-forward network (PFFN). SG-LRU is our creative module that integrates SAM and LRU to comprehensively capture the sequential relationships among items in the sequence.

1) Embedding Layer: Given a sequence $\mathcal{S}_u$ and with an item embedding matrix $\mathbf{M} \in \mathbb{R}^{|\mathcal{I}| \times D}$, we obtain the embedding of each item in the sequence by a look-up operation on $\mathbf{M}$ and denote the embedding representation of $\mathcal{S}_u$ by $\mathbf{E}_u \in \mathbb{R}^{N \times D}$, where D is the dimension of the latent space.

To improve our model's sensitivity to positional information, we incorporate positional embedding. Specifically, a learnable positional embedding matrix $\mathbf{P} \in \mathbb{R}^{N \times D}$ is added to the item embedding matrix $\mathbf{E}_u$, which is shared by all sequences. Since the two matrices are randomly initialized, potentially leading to the training process unstable, a dropout operation [49] and a layer normalization operation [50] are then used to stabilize and normalize them

$$\mathbf{X}_u^{(0)} = \text{LayerNorm}(\text{Dropout}(\mathbf{E}_u + \mathbf{P})) \in \mathbb{R}^{N \times D} \quad (1)$$

where $\mathbf{X}_u^{(0)}$ is designated as the output of the 0th layer in our model, for ease of description.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

2) SG-LRU: The gated LRU [17], [51] employs a gate mechanism to determine the influence of the outputs of the preceding layer on the results of the subsequent recurrent layer. However, as discussed in Section I, in the sequential recommendation scenario, it is more important to adopt a comprehensive perspective to balance the utilization of order information. To meet this requirement, SG-LRU introduces an innovative extension to LRU by incorporating SAM, transforming it into a self-attention gated LRU.

For SG-LRU at the lth layer ($1 \leq l \leq L$), we initially process the output of the previous layer, namely $\mathbf{X}_u^{(l-1)}$, through two different linear transformations, resulting in the creation of two parallel branches. In the first branch, a 1-D convolution operation [17], [36] integrated with an SAM is designed to model global information, while de-emphasizing the significance of order information. The second branch employs a 1-D convolution operation in combination with a behavior-dependent linear recurrent unit (BD-LRU) [17], highlighting the influence of the preceding item on the subsequent one. The outputs of these two branches are then fused by element-wise multiplication, effectively enabling the model to capture complex patterns in sequential data with multiple granularities. Consequently, we incorporate self-attention gating to regulate the flow of user behavior information within BD-LRU. Ultimately, the resulting output is projected back to the D-dimensional space by a linear transformation. The above process can be formalized as follows:

$$\mathbf{L}_1 = \mathbf{X}_u^{(l-1)}\mathbf{W}_1 + \mathbf{b}_1 \in \mathbb{R}^{N\times d} \tag{2}$$

$$\mathbf{G} = \text{SAM}(\mathbf{L}_1) + \text{Conv1-D}_1(\mathbf{L}_1) \in \mathbb{R}^{N\times d} \tag{3}$$

$$\mathbf{L}_2 = \mathbf{X}_u^{(l-1)}\mathbf{W}_2 + \mathbf{b}_2 \in \mathbb{R}^{N\times d} \tag{4}$$

$$\mathbf{H} = \text{BD-LRU}(\text{Conv1-D}_2(\mathbf{L}_2)) \in \mathbb{R}^{N\times d} \tag{5}$$

$$\mathbf{L}_3 = (\mathbf{G} \odot \mathbf{H})\mathbf{W}_3 + \mathbf{b}_3 \in \mathbb{R}^{N\times D} \tag{6}$$

$$\mathbf{O}_u = \text{LayerNorm}(\text{Dropout}(\mathbf{L}_3) + \mathbf{X}_u^{(l-1)}) \in \mathbb{R}^{N\times D} \tag{7}$$

where d is the dimension of the latent space (set to 2-D in our experiment), $\mathbf{W}_m \in \mathbb{R}^{D\times d}$, $\mathbf{b}_m \in \mathbb{R}^d$ ($m = 1, 2$), $\mathbf{W}_3 \in \mathbb{R}^{d\times D}$ and $\mathbf{b}_3 \in \mathbb{R}^D$ are learnable parameters of three linear transformations, Conv1-D_1 and Conv1-D_2 are two 1-D convolution operations. Equations (2) and (3) correspond to the first branch, (5) corresponds to the second branch, and (6) and (7) are for their integration. Here, the layer marker l is used only for inputs and outputs, and is omitted for all operations or parameters, although their values are different in different layers. In the following discussion, unless otherwise specified, we omit the layer marker.

Specifically, SAM calculates the attention score for each item in the sequence

$$\text{SAM}(\mathbf{L}_1) = \text{Softmax}\left(\frac{(\mathbf{L}_1\mathbf{W}_Q)(\mathbf{L}_1\mathbf{W}_K)^\top}{\sqrt{d}} + \mathbf{S}\right)(\mathbf{L}_1\mathbf{W}_V) \tag{8}$$

where the projection matrices $\mathbf{W}_Q \in \mathbb{R}^{d\times d}$, $\mathbf{W}_K \in \mathbb{R}^{d\times d}$, $\mathbf{W}_V \in \mathbb{R}^{d\times d}$ are learnable parameters, $\sqrt{d}$ is a scaling factor, and $\mathbf{S} \in \mathbb{R}^{N\times N}$ is the causal mask which ensures that the output at each position in the sequence is solely based on the information available at and prior to that position. The SAM dynamically allocates the "attention" of each item to others by calculating the similarity scores between the item and all others. We can see that the SAM takes into account the dependencies among items from the global view.

As for BD-LRU, it denotes the operation of a BD-LRU, which enables the inputs of each time step to selectively enter the model for iterative computation using behavior-dependent parameters [17]. Let $\mathbf{x}_t \in \mathbb{R}^d$ be the tth row elements of $\mathbf{X} = \text{Conv1-D}_2(\mathbf{L}_2)$, then BD-LRU introduces a behavior-dependent recursive gate and an input gate. These two gates depend on inputs with different user behaviors and interactions, and a sigmoid function normalizes them to $[0, 1]$ for reasonable gating semantics. The above operations are given by the following equations:

$$\mathbf{r}_t = \text{Sigmoid}(\mathbf{W}_4\mathbf{x}_t + \mathbf{b}_4) \in \mathbb{R}^d \tag{9}$$

$$\mathbf{i}_t = \text{Sigmoid}(\mathbf{W}_5\mathbf{x}_t + \mathbf{b}_5) \in \mathbb{R}^d \tag{10}$$

$$\mathbf{a}_t = \text{Exp}(c \cdot \text{Log}(\text{Sigmoid}(\boldsymbol{\lambda})) \odot \mathbf{r}_t) \in \mathbb{R}^d \tag{11}$$

$$\mathbf{h}_t = \mathbf{a}_t \odot \mathbf{h}_{t-1} + \sqrt{1 - \mathbf{a}_t^2} \odot (\mathbf{i}_t \odot \mathbf{x}_t) \in \mathbb{R}^d \tag{12}$$

where $\mathbf{W}_m \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_m \in \mathbb{R}^d$ ($m = 4, 5$) are learnable parameters of two linear transformations, $\mathbf{r}_t$ and $\mathbf{i}_t$ are two gates coming from these two linear transformations on $\mathbf{x}_t$, $\boldsymbol{\lambda} \in \mathbb{R}^d$ is a parameter vector to be learned for the recurrent gate, c is a scalar constant (set to 8 in our experiment) for calculating the scaling factor $\mathbf{a}_t$, and $\odot$ denotes element-wise multiplication [17], [35], [51]. The exponential function with a negative value guarantees that $\mathbf{a}_t$ is in $[0, 1]$, ensuring that the recurrence is stable. At the start of training, $\boldsymbol{\lambda}$ is initialized such that $\mathbf{a}_t$ is uniformly distributed between 0.9 and 0.999 as in [17], [35]. Consequently, the state of the tth step, denoted as $\mathbf{h}_t$, is derived by a weighted summation of the state of the preceding $(t-1)$th step and the input corresponding to the tth step.

In the subsequent step, we integrate the output of BD-LRU and SAM through element-wise multiplication in (6), resulting in a balanced and comprehensive feature representation. This representation emphasizes both item order and temporal data, while also incorporating global contextual insights. In contrast to RecBLR [17], which introduces a linear transformation as an auxiliary branch alongside BD-LRU, our method conceptualizes the alternative branch as a self-attention-based behavior unit. Unlike the linear transformations employed in RecBLR, which solely concentrate on the current item, our approach considers the global information spanning the entire sequence.

3) PFFN: To effectively capture intricate patterns and interactions within a user behavior sequence, a PFFN module is integrated after the SG-LRU module. PFFN is structured as a MLP block equipped with a gating mechanism that branch into two parallel paths. Within one branch, a Gaussian error linear unit (GeLU) [52] is used to learn nonlinearity for enhancing the model's representational ability. These two branches are merged by element-wise multiplication, and then the result is reshaped back to the original dimension by a linear transformation. Mathematically, the PFFN operation can be expressed as follows:

$$\text{PFFN}(\mathbf{O}_u) = (\text{GeLU}(\mathbf{O}_u\mathbf{W}_6) \odot (\mathbf{O}_u\mathbf{W}_7))\mathbf{W}_8 \tag{13}$$

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

where $\mathbf{W}_m \in \mathbb{R}^{D\times 4\text{-}D}$ $(m=6,7)$ and $\mathbf{W}_8 \in \mathbb{R}^{4\text{-}D\times D}$ are learnable parameters of three linear transformations involved.

We incorporate dropout for regularization, a residual connection [53] to facilitate gradient flow and layer normalization to stabilize training. By combining these techniques, we obtain the output of the lth modelling layer

$$\mathbf{X}_u^{(l)} = \text{LayerNorm}(\text{Dropout}(\text{PFFN}(\mathbf{O}_u) + \mathbf{O}_u). \tag{14}$$

4) Prediction Layer: After going through L layers of sequential dependency modelling, the hidden representation of sequence $\mathcal{S}_u$ is generated and denoted by $\mathbf{X}_u^{(L)}$. Its last row $\mathbf{x}_u^{(L)} \in \mathbb{R}^D$ is chosen as the representation of user u. This representation is then multiplied by the item embedding matrix $\mathbf{M}$ to yield the predicted probability vector $\widehat{\mathbf{y}}_u$

$$\widehat{\mathbf{y}}_u = \text{Softmax}(\mathbf{M} \times \mathbf{x}_u^{(L)}) \in \mathbb{R}^{|\mathcal{I}|}. \tag{15}$$

The predicted probability vector $\widehat{\mathbf{y}}_u$ is used as the ranking score to generate recommendations for user u.

C. Model Training

During the training process, we utilize the cross-entropy to calculate the model's recommendation loss, as shown in the following equation:

$$\mathcal{L}_{\text{Rec}}^u = -\text{Log}(\widehat{y}_{ui}) \tag{16}$$

where $i \in \mathcal{I}$ is the index of the ground-truth item, and $\widehat{y}_{ui}$ is the ith element of the predicted output vector $\widehat{\mathbf{y}}_u$. By minimizing (16), the model learns its parameters in such a way as to maximize the predicted probability of observing the ground truth item i.

In sequential recommendation, datasets typically contain inherent noise, which may lead to overfitting of model training. To mitigate this issue, as shown in Fig. 2, two complementary tasks are integrated into our model: perturbation injection and uncertainty scaling, which aim to improve model robustness and output flexibility, respectively.

1) Perturbation Injection: The task aims to perturb the recurrent process by randomly replacing an item representation in each layer with an embedding sampled from a uniform distribution. This random perturbation introduces disturbances within the model, thereby reducing its tendency to overly rely on the inherent noise present in the interaction data, which could otherwise lead to overfitting.

Specifically, we incorporate a forward process within the BD-LRU block in (9)–(12) by defining a threshold θ in the range $[0,1]$ and sampling p from a uniform distribution. The above perturbation process can be represented mathematically by

$$\overline{\mathbf{x}}_t = \begin{cases} U(\mathbf{0},\mathbf{1}), & p \le \theta \\ \mathbf{x}_t, & p > \theta \end{cases} \tag{17}$$

where $U(\mathbf{0},\mathbf{1})$ is a perturbation vector that is of the same dimension as $\mathbf{x}_t$, with each of its components independently sampled from a uniform distribution over the interval $[0,1]$. Equation (17) illustrates that, during the recurrent process, the input $\mathbf{x}_t$ has a probability θ of being replaced by this perturbation vector.

To implement the perturbation on the BD-LRU, we replace $\mathbf{x}_t$ in (9)–(12) with $\overline{\mathbf{x}}_t$. By traversing through all subsequent layers, we derive an alternative sequence representation $\overline{\mathbf{X}}_u^{(L)}$. This representation is then substituted into (15) for the prediction step. Consequently, the ith predicted value within $\overline{\mathbf{y}}_u$, denoted as $\overline{y}_{ui}$, can be used to compute the perturbation injection loss, which is directly associated with the corresponding ground truth item i. The perturbation injection loss is given by

$$\mathcal{L}_{\text{Per}}^u = -\text{Log}(\overline{y}_{ui}). \tag{18}$$

Incorporating perturbation injection loss during the training process enhances the ability of our model to provide reliable and accurate recommendations even in noisy interaction scenarios.

2) Uncertainty Scaling: The uncertainty scaling task operates on a separate path, where the original sequence passes through different embedding layers before being fed into an uncertainty scaling PFFN to derive the predicted values.

Given sequence $\mathcal{S}_u$ of user u, (15) calculates the probability of recommending each item to the user. To incorporate aleatoric uncertainty into these predicted probabilities, analogous to the Gibbs distribution, we extend (15) to a softmax function with scaled input, formulated as follows:

$$\widetilde{\mathbf{y}}_u = \text{Softmax}\left(\frac{1}{\sigma_u^2}(\mathbf{M} \times \mathbf{x}_u^{(L)})\right) \tag{19}$$

where σ_u is a learnable scaling factor parameter (commonly known as *temperature*). This temperature parameter ensures that the input to the softmax function is appropriately adjusted according to its value. Essentially, σ_u^2 controls the degree of uniformity in the distribution, which is directly related to the inherent uncertainty of $\widetilde{\mathbf{y}}_u$. In an extreme case, if σ_u^2 tends towards $+\infty$, $\widetilde{\mathbf{y}}_u$ approaches a uniform distribution characterized by high entropy, signaling a high degree of uncertainty. In this article, to meet the personalization needs of sequential recommendation, σ_u^2 is dynamically determined for each $\mathcal{S}_u$ in contrast to the approach in [45], which uses a common parameter across all sequences.

We use a PFFN and subsequent dropout and layer normalization operations to represent the scaling factor σ_u. Specifically, as shown in Fig. 1(b), the input sequence $\mathcal{S}_u$ is processed through a newly defined item embedding layer. This layer consists of an item embedding matrix $\mathbf{M}_{\text{c}} \in \mathbb{R}^{|\mathcal{I}|\times D}$ and a position embedding $\mathbf{P}_{\text{c}}$, and yields embedding $\mathbf{X}_u^{\text{c}}$ after going through (1). The embedding of the last item in $\mathcal{S}_u$, namely the last row of $\mathbf{X}_u^{\text{c}}$ denoted as $\mathbf{x}_u^{\text{c}}$, is then fed into an uncertainty estimator modeled by a PFFN to derive variance prediction values. To ensure that $\sigma_u = \text{Exp}((1/2)\text{Log}(\sigma_u^2)) > 0$, $\text{Log}(\sigma_u^2)$ is modeled as follows:

$$\begin{aligned}\text{Log}(\sigma_u^2) = \mathbf{M}_{\text{c}}[i,:]^\top \\ \times \text{LayerNorm}(\text{Dropout}(\text{PFFN}(\mathbf{x}_u^{\text{c}}) + \mathbf{x}_u^{\text{c}}))\end{aligned} \tag{20}$$

where i is the index of the ground truth item and $\mathbf{M}_{\text{c}}[i,:] \in \mathbb{R}^D$ is the ith row of $\mathbf{M}_{\text{c}}$.

After obtaining σ_u, we derive the predicted probability $\widetilde{\mathbf{y}}_u$, which incorporates the aleatoric uncertainty. Analogous to (16),

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

we can then calculate the corresponding uncertainty incorporated recommendation loss by

$$\mathcal{L}^u_{\text{Unc}} = -\text{Log}(\tilde{y}_{ui}) \tag{21}$$

where $\tilde{y}_{ui}$ is the predicted value associated with the ground truth item i from (19). Integrating the insights from the study in [45] into our framework, we can relate $\mathcal{L}^u_{\text{Unc}}$ to $\mathcal{L}^u_{\text{Rec}}$ in (16) as follows:

$$\begin{aligned}\mathcal{L}^u_{\text{Unc}} &\approx -\frac{1}{\sigma_u^2}\text{Log}(\widehat{y}_{ui}) + \beta\text{Log}(\sigma_u) \\ &= \frac{1}{\sigma_u^2}\mathcal{L}^u_{\text{Rec}} + \beta\text{Log}(\sigma_u).\end{aligned} \tag{22}$$

In (22), the first term emphasizes that the recommendation loss is inversely weighted by σ_u^{-2}, implying that a larger σ_u reduces the importance of the recommendation loss. The second term of the equation introduces a penalty mechanism that acts on the weight if σ_u becomes too large, with β acting as the modulating factor for this penalty. Thus, by incorporating aleatoric uncertainty into the recommendation modelling architecture, the uncertainty estimator attempts to alleviate the challenge posed by inherent noise.

3) Optimization: For the sequential behavior of user u and the corresponding ground truth item i, we unify the above three losses into a learning framework. Mathematically, the joint learning objective is formulated as follows:

$$\begin{aligned}\mathcal{L}^u &= \mathcal{L}^u_{\text{Rec}} + \mathcal{L}^u_{\text{Per}} + \alpha\mathcal{L}^u_{\text{Unc}} \\ &= \left(1 + \frac{\alpha}{\sigma_u^2}\right)\mathcal{L}^u_{\text{Rec}} + \mathcal{L}^u_{\text{Per}} + \alpha\beta\text{Log}(\sigma_u)\end{aligned} \tag{23}$$

where α is an auxiliary positive hyperparameter to assist σ_u in regulating the uncertainty effect. This positive hyperparameter ensures that as σ_u^2 increases, the contribution of $\mathcal{L}^u_{\text{Rec}}$ decreases. In (23), $\mathcal{L}^u_{\text{Rec}}$ retains its effectiveness even as σ_u^2 tends to $+\infty$. The regularizing effect of $\text{Log}(\sigma_u)$ can be considered negligible, so we set β to 0 for streamlining the hyperparameter configuration. Extending this to sequences across all users, the final loss function is summarized as follows:

$$\mathcal{L} = \sum_{u\in\mathcal{U}}\left(\left(1 + \frac{\alpha}{\sigma_u^2}\right)\mathcal{L}^u_{\text{Rec}} + \mathcal{L}^u_{\text{Per}}\right). \tag{24}$$

IV. Experiment and Analysis

To validate the performance of our SLSRec model, we conduct several experiments on five real-world datasets. Our objectives are to compare its performance with twelve state-of-the-art baseline models and to perform a thorough model analysis. In essence, we attempt to answer the following four research questions (RQs).

1) *RQ1:* How does SLSRec perform against various outstanding baselines?
2) *RQ2:* How do the key modules in SLSRec affect its overall performance?
3) *RQ3:* How does SLSRec perform on noisy interactions or long-tail items?
4) *RQ4:* How do hyperparameters affect the performance of SLSRec?

TABLE I
Dataset Statistics After Preprocessing

Dataset	# Users	# Items	# Interactions	Aver. Length	Density
ML-1M	6040	3416	1.0 M	165.5	5e-2
Beauty	52 204	57 289	0.4 M	7.6	1e-4
Video	31 013	23 715	0.3 M	9.3	4e-4
Sports	83 970	83 728	0.6 M	7.1	8e-5
Steam	334 730	13 047	3.7 M	11.0	8e-4

A. Experimental Setup

1) Datasets: We implement experiments and evaluate our model using the following five real-world datasets.

a) *MovieLens-1M:* A benchmark dataset commonly used for movie recommendation tasks. We utilize the widely adopted ML-1M version here [54].
b) *Amazon:* A collection of datasets featuring product reviews sourced from Amazon. Specifically, we select the Beauty, Video and Sports categories for our analysis [55], [56].
c) *Steam:* A dataset of video game reviews obtained from Steam, a prominent online video game distribution platform [13].

For each dataset, interaction records are grouped by user, and then sorted by ascending timestamp to form the interaction sequence for each user. To reduce the workload of the experiment and to ensure a fair comparison, we closely follow the configurations outlined in the LRURec work [16] and request experimental datasets directly from the authors. In particular, these datasets are preprocessed using the procedures outlined in [13], [14], [57], [58] to construct input sequences. Users and items with fewer than five interactions are excluded, and the maximum sequence length is set to 200 for ML-1M and 50 for the other four datasets. These datasets are partitioned using the leave-last-out strategy, where the most recent item is assigned to the test set, the next most recent item is assigned to the validation set, and all other items in the sequences together form the training set. During the testing phase, both training and validation items are included as part of the input. The statistics of these datasets after preprocessing are shown in Table I.

2) Baseline Methods: We evaluate the performance of our SLSRec model by benchmarking it against a comprehensive set of baseline methods, including traditional factorization and Markov chain techniques (MF, FISM and FPMC), RNN-based architectures (GRU4Rec and NARM), Transformer-based or MLP-based models (SASRec, BERT4Rec and FMLP-Rec), an uncertainty aware method (STOSA) and LRU-based models (LRURec and RecBLR). A brief overview of these methods is given below.

a) *MF [27]:* A classical factorization model that infers latent representations for users and items to predict next-item interactions.
b) *FISM [28]:* Rather than explicitly modelling user preferences, FISM predicts subsequent interactions by exploiting item-to-item similarity.
c) *FPMC [7]:* A matrix factorization model that employs Markov chains to capture user transition patterns.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

d) *GRU4Rec [8], [9]:* A conventional sequential recommendation method that uses a GRU-based architecture to model user-item interactions.
e) *NARM [32]:* An RNN-based sequential recommender that integrates local and global user modelling for accurate next-item predictions.
f) *SASRec [13]:* As a pioneering Transformer-based sequential recommendation model, SASRec employs unidirectional SAMs with the causal mask to model user-item transition patterns.
g) *BERT4Rec [33]:* A bidirectional SAM-based Transformer without the causal mask for sequential recommendation, trained by predicting masked items within a random subset.
h) *FMLP-Rec [25]:* A filter-based, all-MLP model operating in the complex domain that utilizes fast Fourier transform for efficient learning.
i) *STOSA [38]:* A sequence recommendation model that captures dynamic uncertainty and collaborative transitivity using stochastic embeddings and Wasserstein self-attention.
j) *BSARec [59]:* A sequential recommendation model based on self-attention that leverages Fourier transforms to introduce inductive bias and integrate low/high-frequency information, effectively addressing the oversmoothing issue in Transformer-based SR and significantly improving recommendation performance.
k) *LRURec [16]:* A sequential recommendation model based on LRU that combines efficient inference with parallelizable training via recursive parallelization.
l) *RecBLR [17]:* A gated behavior-dependent recurrent model that uses real-valued LRU and a hardware-aware parallel scanning algorithm.

3) Evaluation Metrics: We use two standard metrics, Recall (Recall@K) and normalized discounted cumulative gain (NDCG@K) with $K \in \{10, 20\}$, to evaluate the performance of a model. For unbiased evaluation, we rank predicted items relative to all items in the dataset to calculate the metric scores.

4) Implementation Details: For all baseline models, we strictly follow the protocols detailed in the original publications for their respective implementations [16], [17], [38]. All of them except for STOSA are implemented using the LRURec code repository, with their parameters configured identically to those in LRURec [16], [17]. Specifically, each model is trained using the cross-entropy loss function, employing the AdamW optimizer with a constant learning rate of 1e-3 and a weight decay of 1e-2. During the training process, the embedding dimension remains fixed at 64, a batch size of 128 is utilized, and training is limited to a maximum of 1000 epochs. Validation is conducted every 5 epochs, and an early stopping mechanism is activated if there is no improvement in the validation Recall@10 metric for 10 consecutive validation epochs. The dropout rate is selected from the set $\{0.2, 0.4, 0.6, 0.8\}$.

For STOSA, to make a fair comparison, we follow its original parameter selection strategy. The embedding dimension, the maximum sequence length, the learning rate, the $L2$ regularization weight and the dropout rate are chosen from $\{32, 64\}$, $\{50, 200\}$, $\{1e\text{-}4, 1e\text{-}3\}$, $\{1e\text{-}3, 1e\text{-}2, 1e\text{-}1\}$ and $\{0.3, 0.5, 0.7\}$, respectively.

For our SLSRec model, its implementation is also based on the LRURec code repository. All common parameters are configured identically to the above baselines, except for the dropout rate. In response to the complex characteristics of the various datasets, we adopt a more comprehensive hyperparameter search strategy for the dropout rate. Specifically, we tune it in $\{0, 0.2, 0.4, 0.6, 0.8\}$ for ML-1M, Steam and Video, and in $\{0, 0.25, 0.5, 0.75, 0.95\}$ for Beauty and Sports, respectively. This approach allows us to tailor the dropout rate to the unique characteristics of each dataset, thereby improving the model's performance across different data distributions. The model specific hyperparameters of SLSRec, the perturbation rate θ and the uncertainty parameter α, are chosen from $\{0.05, 0.1, 0.15, 0.2, 0.25\}$ and $\{0.01, 0.05, 0.1, 0.3, 0.5, 0.7, 1\}$, respectively.

B. Overall Performance (RQ1)

As the experimental datasets, including their training/validation/test segmentation, are the same as those used in [16], [17], we directly use the reported results of these studies for comparison. Our primary results are summarized in Table II, where the best results are bolded and the second-best results are underlined for clarity. Additionally, we calculate the relative improvement of our proposed SLSRec over the best performing baseline, denoted as "Improv." in Table II. From the table, we can make the following observations.

1) Traditional methods exhibit comparatively suboptimal performance. MF, FISM, and FPMC rely on matrix decomposition or Markov chains to capture sequential patterns, which are limited to capturing only short-term dependencies, ignoring long-term dependencies. They are therefore limited to identifying only low-order, simple dependencies.
2) RNN-based methods outperform traditional models in performance. In particular, GRU4Rec and NARM model sequences by progressively updating their hidden states, and uniquely incorporate a gating mechanism that enables them to capture longer dependencies compared to FISM and FPMC.
3) Transformer-based or MLP-based methods beat RNN-based models. SASRec and BERT4Rec use SAMs and FMLP-Rec uses a fully connected MLP layer to capture sequence interactions, and they capture global information and emphasize those interactions that are truly relevant. Remarkably, integrating a frequency filter with SAMs enables BSARec to secure the second-best performance on the Sports dataset, highlighting the significant impact of combining SAMs with other models in sequential recommendation.
4) LRU-based methods outperform previous Transformer-based and MLP-based models. LRURec and RecBLR use state space model-based LRU to model sequences, ensuring efficient parallelization of both training and inference. In addition, they incorporate residual connections and PFFNs to further enhance the representation capabilities of these models.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
PERFORMANCE COMPARISON OF THE PROPOSED SLSREC MODEL AGAINST BASELINE MODELS ON DATASETS INCLUDING ML-1M, BEAUTY, VIDEO, SPORTS AND STEAM

Dataset	Metric	MF	FISM	FPMC	GRU4Rec	NARM	SASRec	BERT4Rec	FMLP-Rec	STOSA	BSARec	LRURec	RecBLR	SLSRec	Improv.
ML-1M	Recall@10	0.06290	0.07368	0.23030	0.28317	0.27290	0.31441	0.30894	0.29073	0.24226	0.27353	0.32619	0.34626	**0.35595**	2.80%
	Recall@20	0.10910	0.12520	0.31510	0.39378	0.37066	0.43084	0.41738	0.39454	0.32922	0.37906	0.43313	0.44979	**0.46732**	3.90%
	NDCG@10	0.02835	0.03327	0.12030	0.15943	0.15310	0.18293	0.16377	0.15295	0.14436	0.15529	0.19292	0.20715	**0.21332**	2.98%
	NDCG@20	0.03991	0.04655	0.14370	0.18735	0.17774	0.21232	0.19118	0.17912	0.16611	0.18182	0.21988	0.23385	**0.24141**	3.23%
Beauty	Recall@10	0.02596	0.02614	0.03586	0.03379	0.03312	0.05043	0.04679	0.04623	0.03818	0.05330	0.05249	0.05345	**0.05601**	4.79%
	Recall@20	0.03987	0.04008	0.04699	0.04751	0.04708	0.06965	0.06727	0.06336	0.05355	0.07213	0.07202	0.07329	**0.07727**	5.43%
	NDCG@10	0.01193	0.01281	0.02091	0.01997	0.01887	0.02917	0.02422	0.02515	0.02169	0.03076	0.03058	0.03105	**0.03252**	4.73%
	NDCG@20	0.01563	0.01621	0.02371	0.02343	0.02239	0.03403	0.02937	0.02945	0.02529	0.03598	0.03551	0.03606	**0.03790**	5.10%
Video	Recall@10	0.02634	0.05815	0.08110	0.09512	0.09777	0.10910	0.11252	0.10885	0.07613	0.11686	0.11271	0.11725	**0.11748**	0.20%
	Recall@20	0.04052	0.09281	0.11830	0.13959	0.14488	0.15989	0.16640	0.15991	0.11553	0.17188	0.16638	0.17202	**0.17353**	0.88%
	NDCG@10	0.01337	0.02813	0.04318	0.05081	0.05329	0.05829	0.05600	0.05294	0.04028	0.06284	0.06061	0.06367	**0.06370**	0.05%
	NDCG@20	0.01722	0.03692	0.05267	0.06200	0.06512	0.07111	0.06958	0.06576	0.05907	0.07685	0.07409	0.07748	**0.07783**	0.45%
Sports	Recall@10	0.00413	0.01614	0.01936	0.01882	0.02087	0.02294	0.02879	0.02987	0.02281	0.03218	0.03142	0.03067	**0.03356**	4.29%
	Recall@20	0.00861	0.02644	0.02643	0.02841	0.03265	0.03428	0.04295	0.04276	0.03341	0.04536	0.04484	0.04368	**0.04768**	5.11%
	NDCG@10	0.00266	0.00810	0.01070	0.01002	0.01117	0.01236	0.01457	0.01546	0.01307	0.01855	0.01810	0.01762	**0.01918**	3.40%
	NDCG@20	0.00362	0.01036	0.01257	0.01243	0.01414	0.01520	0.01813	0.01871	0.01573	0.02208	0.02147	0.02088	**0.02273**	2.94%
Steam	Recall@10	0.12166	0.08561	0.08773	0.12204	0.12166	0.12597	0.12188	0.11987	0.08793	0.11472	0.12743	0.13040	**0.14089**	8.04%
	Recall@20	0.18199	0.13450	0.13570	0.18319	0.18199	0.18716	0.18245	0.17719	0.13908	0.17358	0.18905	0.19305	**0.20697**	7.21%
	NDCG@10	0.06480	0.04209	0.04377	0.06512	0.06480	0.06759	0.06433	0.06081	0.04488	0.06120	0.06850	0.07046	**0.07640**	8.43%
	NDCG@20	0.07996	0.05439	0.05582	0.08050	0.07996	0.08298	0.08117	0.07521	0.05774	0.07605	0.08401	0.08623	**0.09245**	7.21%

Note: Best results on each dataset are highlighted in bold, while the second-best results are underlined.

5) SLSRec consistently surpasses all baseline methods across all evaluation metrics and datasets, demonstrating remarkable superiority. Specifically, SLSRec achieves an average performance improvement of 4.06% over the best baseline methods, highlighting the effectiveness of our approach. This success can be attributed to the appropriate utilization of item order relationships within sequences, which are either neglected by SAM-based models or overemphasized by LRU-based models.

C. Ablation Study (RQ2)

The main contributions of SLSRec lie in the seamless integration of a self-attention component and an LRU component, coupled with perturbation injection and uncertainty scaling techniques, which together facilitate effective learning of appropriate integration. To investigate the specific impact of these components on the performance of SLSRec, we conduct a comprehensive ablation study. In this study, customized variants of SLSRec, each with a different modification, are evaluated: *w/o SAM* excludes the self-attention layer within the SG-LRU module, *w/o BD-LRU* removes the BD-LRU layer from the SG-LRU module, *w/o P* neglects the perturbation injection task, *w/o U* omits the uncertainty scaling task, and *w/o P&U* simultaneously removes perturbation injection and uncertainty scaling tasks, *i.e.*, it is the simple combination of SAM and LRU. The comparative performance of SLSRec and its modified variants is shown in Table III. From this table, the following observations can be made.

1) Each component plays a critical role in boosting the overall effectiveness of SLSRec, with one exception: w/o BD-LRU produces an increase in NDCG@20 for Video. In contrast, the removal of any other component leads to a decrease in performance across different datasets, albeit to varying degrees.
2) The influence of individual components varies depending on the specific characteristics of the dataset. A notable observation is the significant average performance drop of 10.95% for the sparse Sports when the uncertainty scaling task is excluded. This demonstrates the importance of incorporating a mechanism to deal with missing data through uncertainty scaling, especially for sparse datasets where it is particularly critical.
3) Simultaneously removing the perturbation injection and uncertainty scaling tasks at the same time leads to varying degrees of degradation in the recommended performance for all datasets, which is worse than LRURec and RecBLR using only separate architectures. This confirms our argument that simple integration of SAM and LRU (*i.e.*, w/o P&U) may lead to overfitting, and highlights the need for our design of two complementary tasks.

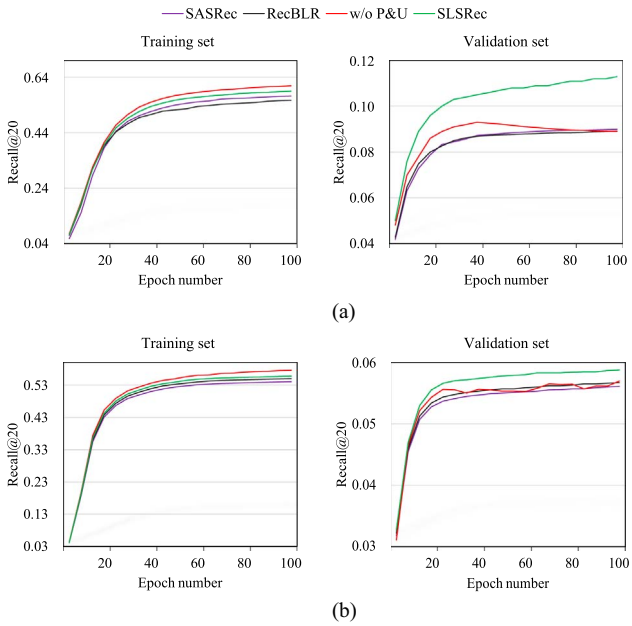


Fig. 3. Learning curves on the training and validation sets for SASRec, RecBLR, the fused model (w/o P&U) and SLSRec on: (a) beauty; and (b) sports.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

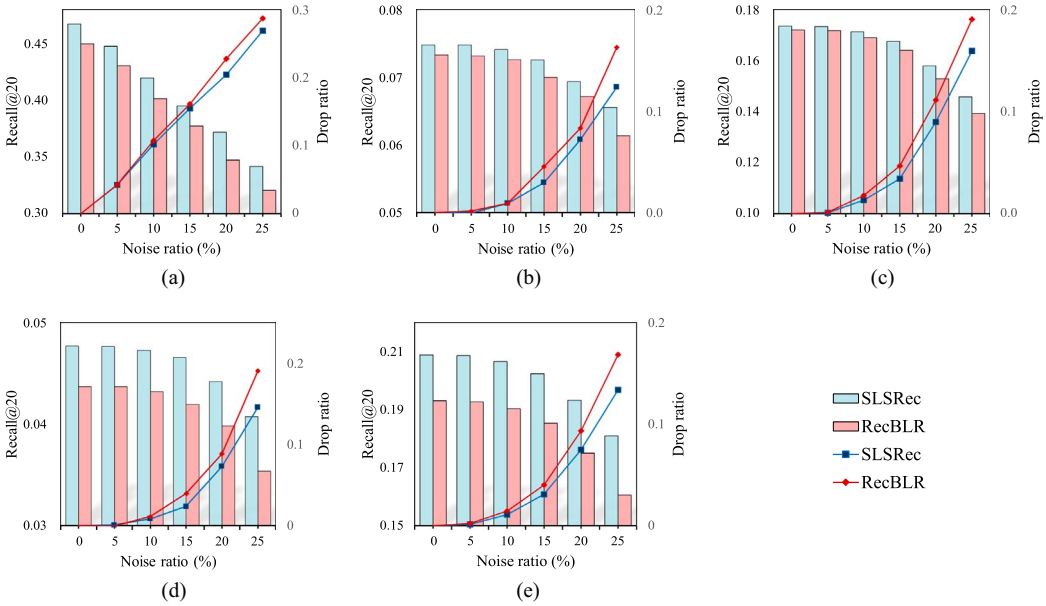


Fig. 4. Performance of SLSRec and RecBLR *w.r.t.* different noise ratios on five datasets. The bar chart represents the performance in Recall@20 and the line chart shows the corresponding drop ratio. (a) ML-1M. (b) Beauty. (c) Video. (d) Sports. (e) Steam.

TABLE III
RESULTS OF ABLATION STUDIES ON RECALL@20 AND NDCG@20 (DROP PERCENTAGE COMPARED TO SLSREC)

Variants	Metrics	ML-1M	Beauty	Video	Sports	Steam
SLSRec	Recall@20	0.46732	0.07727	0.17353	0.04768	0.20697
	NDCG@20	0.24141	0.03790	0.07782	0.02273	0.09245
w/o SAM	Recall@20	0.46123 (↓1.30%)	0.07248 (↓6.20%)	0.17197 (↓0.90%)	0.04731 (↓0.78%)	0.20427 (↓1.30%)
	NDCG@20	0.23863 (↓1.15%)	0.03601 (↓4.99%)	0.07663 (↓1.53%)	0.02214 (↓2.60%)	0.09134 (↓1.20%)
w/o BD-LRU	Recall@20	0.45815 (↓1.96%)	0.07084 (↓8.32%)	0.17139 (↓1.23%)	0.04564 (↓4.28%)	0.19924 (↓3.73%)
	NDCG@20	0.23601 (↓2.24%)	0.03518 (↓7.18%)	0.07887 (↑1.35%)	0.02144 (↓5.68%)	0.08800 (↓4.81%)
w/o P	Recall@20	0.45803 (↓1.99%)	0.07362 (↓4.72%)	0.17261 (↓0.53%)	0.04694 (↓1.55%)	0.20351 (↓1.67%)
	NDCG@20	0.23503 (↓2.64%)	0.03627 (↓4.30%)	0.07746 (↓0.46%)	0.02185 (↓3.87%)	0.09096 (↓1.61%)
w/o U	Recall@20	0.45987 (↓1.59%)	0.07584 (↓1.85%)	0.17306 (↓0.27%)	0.04309 (↓9.63%)	0.20595 (↓0.49%)
	NDCG@20	0.23698 (↓1.84%)	0.03682 (↓2.85%)	0.07662 (↓1.54%)	0.01994 (↓12.27%)	0.09209 (↓0.39%)
w/o P&U	Recall@20	0.44852 (↓4.02%)	0.07345 (↓4.94%)	0.17217 (↓0.78%)	0.04436 (↓6.96%)	0.18744 (↓9.44%)
	NDCG@20	0.22980 (↓4.81%)	0.03589 (↓5.30%)	0.07551 (↓2.97%)	0.02085 (↓8.27%)	0.08142 (↓11.93%)

We also find that in most cases, removing these two complementary tasks is worse than removing any single module, except for Sports, where removing uncertain scaling causes the greatest performance degradation. This may be due to the fact that data sparsity has a greater impact on Sports. Therefore, the recommendation accuracy can be significantly improved by using the uncertainty scaling task alone.

As mentioned in Section I, to further demonstrate how the simple integration of SAM and LRU architectures (w/o P&U) leads to overfitting, Fig. 3 gives learning curves on the training and validation sets for SASRec, RecBLR, the fused model (w/o P&U) and SLSRec on Beauty and Sports with Recall@20 as the evaluation metric. As shown in the figure, the fused model (w/o P&U, the red curve) achieves the best performance on the training set, significantly outperforming both SASRec and RecBLR. Conversely, its validation performance fluctuates or degrades as training progresses, particularly after 40 epochs, and occasionally falls below that of SASRec or RecBLR. This indicates that the fused model tends to overfit spurious features in the training data and consequently suffers from limited generalization. In contrast, the proposed SLSRec maintains robust and stable performance on the validation set throughout the entire training process. This observation not only highlights the superiority of SLSRec but also underscores the effectiveness of its dual-pronged anti-overfitting strategy in alleviating overfitting and enhancing generalization.

In summary, the ablation results conclusively validate the effectiveness of all the proposed components and the carefully designed architecture of SLSRec in delivering robust sequential

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

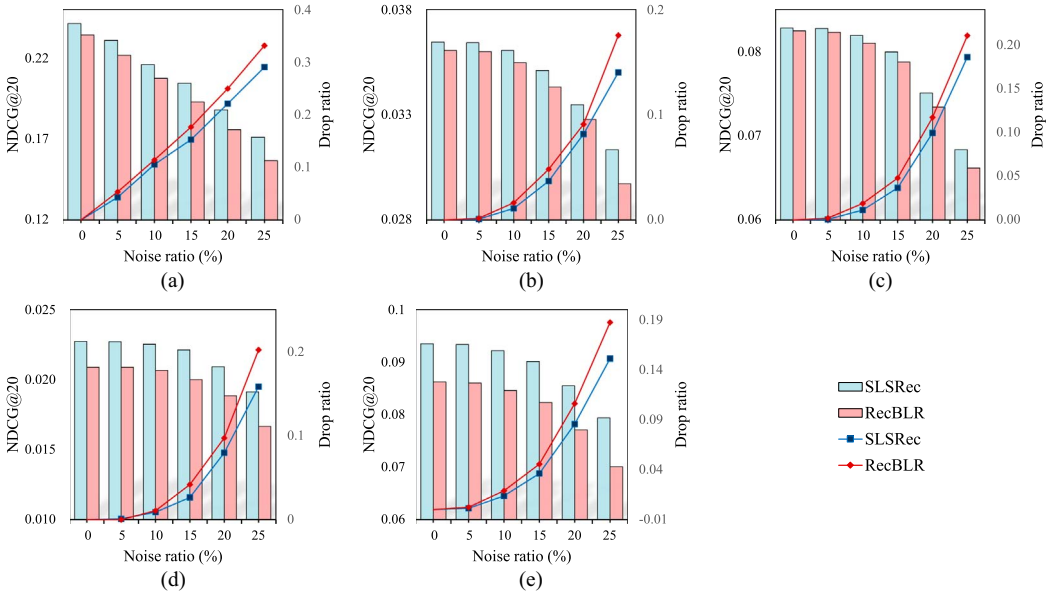


Fig. 5. Performance of SLSRec and RecBLR *w.r.t.* different noise ratios on five datasets. The bar chart represents the performance in NDCG@20 and the line chart shows the corresponding drop ratio. (a) ML-1M. (b) Beauty. (c) Video. (d) Sports. (e) Steam.

recommendations across a variety of data scenarios. This highlights the critical role that each component plays in the overall success of the model.

D. Evaluation on Noise and Long-Tail Scenarios (RQ3)

1) Noisy Interaction Robustness: To evaluate the resilience of the SLSRec model against noisy interactions, we introduce different levels of noise {5%, 10%, 15%, 20%, 25%} into the user-item interaction sequences within the test sets, while keeping the training sets unchanged. Specifically, we randomly replace a certain number of items within each sequence according to the prescribed noise ratio, mimicking real-world scenarios where users may inadvertently interact with unintended items. We chose RecBLR to compare the performance of our SLSRec model because it is a strong backbone of our model and has the most competitive recommendation performance in Table II. Figs. 4 and 5 show their results in terms of Recall@20 and NDCG@20, respectively.

From the analysis of these two figures, it is clear that the introduction of noise into the test set leads to varying degrees of performance degradation for both models. Specifically, the degree of degradation increases as the amount of noise increases, but these decreases follow approximately linear patterns. The linearity of these patterns suggests that the underlying architectures of both models are relatively resilient to some certain levels of noise. In addition, it can be observed that SLSRec outperforms RecBLR across all metrics, all datasets and all noise levels. It is worth noting that the rate of decline is lower for SLSRec in all cases. These observations demonstrate the importance of incorporating self-attention gate, perturbation injection and uncertainty scaling strategies into our model. By integrating these components, we can significantly enhance SLSRec's ability to effectively manage noise across a wide range of datasets.

TABLE IV
PERFORMANCE COMPARISON ON LONG-TAIL ITEMS ACCORDING TO TAIL METRICS

Dataset	Metric	LRURec	RecBLR	SLSRec	Improv.
ML-1M	Recall@20	0.24109	0.25014	**0.25889**	3.50%
	NDCG@20	0.12192	0.12929	**0.13390**	3.57%
Beauty	Recall@20	0.02016	0.02005	**0.02210**	9.62%
	NDCG@20	0.01081	0.01086	**0.01171**	7.83%
Video	Recall@20	0.05760	**0.05854**	0.05852	−0.03%
	NDCG@20	0.02566	**0.02661**	0.02617	−1.65%
Sports	Recall@20	0.01203	0.01182	**0.01351**	12.30%
	NDCG@20	0.00611	0.00624	**0.00687**	10.10%
Steam	Recall@20	0.07077	0.07251	**0.08078**	11.41%
	NDCG@20	0.03233	0.03333	**0.03744**	12.33%

Note: Best results for each metric on each dataset are highlighted in bold, while the second-best results are underlined.

2) Long-Tail Recommendation: To evaluate SLSRec's recommendation performance on long-tail items, we implement an experiment following the setup in [45], [60]. Specifically, tail items are defined as those with fewer interactions and accounting for 50% of all interactions in the dataset, with the remainder being head items. Using this threshold value, 86.8%, 92%, 92.1%, 92.1%, and 97.1% of items in ML-1M, Beauty, Video, Sports and Steam are tail items, respectively. As in [60], this protocol focuses on the ranking of tail items. We still conduct a full sorting of items and select only tail items from the test set as ground truth. We then calculate tail Recall@20 and NDCG@20 metrics by assigning a value of 1 if a test item appears in the top-20 results, and a value of 0 otherwise.

Experimental results for long-tail recommendation are shown in Table IV. Across all datasets, SLSRec consistently stands out

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE V
EFFECT OF UNCERTAINTY RATE α

	α	0.01	0.05	0.1	0.3	0.5	0.7	1
ML-1M	Recall@20	0.45646	0.46553	0.46464	**0.46732**	0.45482	0.45348	0.45960
	NDCG@20	0.23732	0.24006	0.23887	**0.24141**	0.23423	0.23196	0.23474
Beauty	Recall@20	**0.07727**	0.07655	0.07683	0.07477	0.07601	0.07624	0.07559
	NDCG@20	**0.03790**	0.03705	0.03746	0.03646	0.03698	0.03705	0.03688
Video	Recall@20	0.17139	0.17295	0.17239	0.17353	0.17129	**0.17387**	0.17273
	NDCG@20	0.07682	0.07790	0.07727	0.07783	0.07728	**0.07832**	0.07814
Sports	Recall@20	0.04829	**0.04834**	0.04726	0.04768	0.04793	0.04746	0.04731
	NDCG@20	**0.02300**	0.02284	0.02255	0.02273	0.02274	0.02185	0.02229
Steam	Recall@20	0.20718	0.20767	0.20654	0.20697	0.20674	0.20637	**0.20844**
	NDCG@20	0.09292	0.09285	0.09215	0.09245	0.09242	0.09243	**0.09309**

Note: Best results for each metric on each dataset across different α values are highlighted in bold.

as a superior performer, achieving either optimal or suboptimal results. It outperforms the suboptimal RecBLR by an average of 6.90%, with performance gains ranging from 3.5% to 12.33% across all datasets except Video. Notably, for Sports and Steam, SLSRec achieves at least 10.10% performance improvement, and for the suboptimal case, the difference between SLSRec and the optimal RecBLR is negligible. We attribute the good performance of SLSRec on long-tail items to its incorporation of aleatoric uncertainty scaling, which helps to assign a higher degree of variance to instances where positive samples cannot be definitively identified (long-tail items).

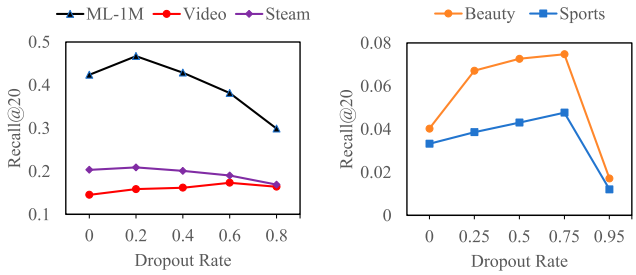


Fig. 6. Influence of the dropout rate on Recall@20.

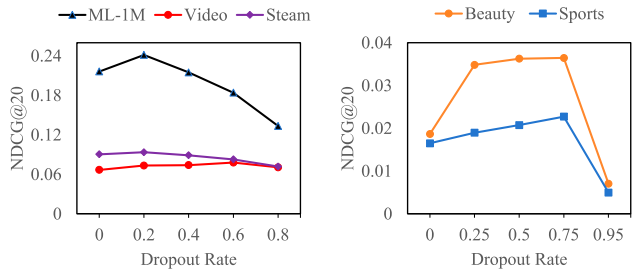


Fig. 7. Influence of the dropout rate on NDCG@20.

E. Hyperparameter Sensitivity (RQ4)

We perform an extensive series of hyperparameter experiments on five different datasets to assess the sensitivity of our SLSRec model to variations in dropout rate, perturbation rate, and uncertainty parameter. The purpose of these experiments is to explore the optimal configuration of these hyperparameters and to gain insight into their positive impact on model performance.

1) Dropout Rate: The dropout rate controls how many neurons in the neural network are randomly discarded, reducing the model's dependence on specific training samples and thus reducing the risk of overfitting. This forces the network to learn more robust features that generalize better across different data samples. We conduct experiments with different dropout rates across multiple datasets. Figs. 6 and 7 illustrate a comparison of the model's performance under different hyperparameter configurations. From these figures, we can see that the optimal performance is achieved at a dropout rate of 0.2 for dense datasets, such as ML-1M, and at a dropout rate of 0.75 for sparse datasets, such as Sports. This suggests that the appropriate dropout rate is dataset dependent and that fine-tuning this hyperparameter is essential to maximize the effectiveness of the model.

2) Perturbation Rate: The perturbation ratio θ in (17) controls the amount of perturbation introduced into the training set during the training phase. We fine-tune it within a range of 0.05 to 0.25 to investigate its effect on the performance of SLSRec. As shown in Table V, the performance of the SLSRec shows a noticeable trend as the hyperparameter θ varies. In particular, the optimal performance is observed at $\theta = 0.1$ for most datasets. Conversely, for Video and Sports, the peak performance occurs at different θ values, suggesting that these datasets benefit from a higher level of perturbation injection during training. Therefore, for practical applications, we use a default perturbation rate of 0.1 in our experimental setup.

3) Uncertainty Parameter: The uncertainty parameter α in (24) modulates the confidence level of the model in its current predictions. As illustrated in Table VI, we implement experiments using seven different α values, ranging from 0.01 to 1. These results show that the influence of the uncertainty rate on the model's recommended performance is more subtle compared to other hyperparameters. In particular, the optimal value of α varies across different datasets: $\alpha = 0.3$ for ML-1M, $\alpha = 0.01$ for Beauty and $\alpha = 1$ for Steam. This variability in the optimal α value demonstrates the importance of dataset specific hyperparameter tuning, which can be attributed to the different

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE VI
EFFECT OF PERTURBATION RATE θ

	θ	0.05	0.1	0.15	0.2	0.25
ML-1M	Recall@20	0.46265	**0.46732**	0.46008	0.46151	0.46188
	NDCG@20	0.23799	**0.24141**	0.23170	0.23936	0.23842
Beauty	Recall@20	0.07623	**0.07727**	0.07522	0.07596	0.07380
	NDCG@20	0.03718	**0.03790**	0.03645	0.03682	0.03588
Video	Recall@20	0.17234	0.17353	**0.17573**	0.17524	0.17518
	NDCG@20	0.07757	0.07783	0.07882	**0.07956**	0.07835
Sports	Recall@20	0.04784	0.04768	**0.04794**	0.04649	0.04581
	NDCG@20	0.02248	**0.02273**	0.02251	0.02253	0.02163
Steam	Recall@20	0.20644	**0.20697**	0.20556	0.20621	0.20678
	NDCG@20	0.09198	**0.09245**	0.09168	0.09228	0.09237

Note: Best results for each metric on each dataset across different θ values are highlighted in bold.

degrees of sparsity and unique characteristics inherent in each dataset.

Therefore, choosing the appropriate value for the hyperparameter α requires careful consideration and a thorough understanding of the dataset characteristics. To comprehensively evaluate our performance, we adopt a tailored approach for each dataset, setting the uncertainty rate to 0.01 specifically for Beauty and to 0.3 for all other datasets. These values are determined according to the optimal performance observed in our preliminary experiments.

V. CONCLUSION

To properly exploit item order information within historical interaction sequences, we propose a sequential recommendation model based on self-attention gated LRU, called SLSRec. In terms of its model structure, SLSRec integrates different modelling techniques, using SAMs and LRUs in a carefully crafted manner. In terms of technical skill, SLSRec incorporates perturbation injection and uncertainty scaling strategies to tackle the pervasive problem of noisy data in recommender systems. By synergizing its sophisticated model structure with advanced technical skills, SLSRec excels at capturing sequential dependencies, thereby generating highly accurate recommendations. It also demonstrates robustness in mitigating the impact of noisy transactions and in dealing with the long-tail effect. Extensive experiments conducted on real-world datasets underscore SLSRec's superior recommendation performance, outperforming a number of state-of-the-art baseline models that are based on Transformer, LRU and uncertainty modelling.

Future research could focus on exploring more strategies for exploiting sequential relationships, such as direct modelling or the concatenation of SAM and LRU. This could allow a more refined use of the item order information and further improve the model's recommendation performance. Another promising direction of investigation would be to develop a more efficient uncertainty estimator and combine it with contrastive learning, to further improve the long-tail effectiveness and robustness of the model.

REFERENCES

[1] H. Wang et al., "Preliminary investigation of alleviating user cold-start problem in e-commerce with deep cross-domain recommender system," in *Companion Proc. World Wide Web Conf. (WWW)*, 2019, pp. 398–403.

[2] M. Aivazoglou et al., "A fine-grained social network recommender system," *Soc. Netw. Anal. Min.*, vol. 10, no. 1, p. 8, 2020.

[3] M. Li, Y. Zhang, W. Zhang, S. Zhao, X. Piao, and B. Yin, "CSAT: Contrastive sampling-aggregating transformer for community detection in attribute-missing networks," *IEEE Trans. Comput. Soc. Syst.*, vol. 11, no. 2, pp. 2277–2290, Apr. 2024.

[4] Z. Z. Darban and M. H. Valipour, "GHRS: Graph-based hybrid recommendation system with application to movie recommendation," *Expert Syst. Appl.*, vol. 200, 2022, Art. no. 116850.

[5] V. Moscato, A. Picariello, and G. Sperlí, "An emotional recommender system for music," *IEEE Intell. Syst.*, vol. 36, no. 5, pp. 57–68, Sep./Oct. 2021.

[6] R. He and J. J. McAuley, "Fusing similarity models with Markov chains for sparse sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2016, pp. 191–200.

[7] S. Rendle, C. Freudenthaler, and L. Schmidt-Thieme, "Factorizing personalized Markov chains for next-basket recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 811–820.

[8] B. Hidasi and A. Karatzoglou, "Recurrent neural networks with top-k gains for session-based recommendations," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2018, pp. 843–852.

[9] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proc. 4th Int. Conf. Learn. Representations (ICLR),* 2016.

[10] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.

[11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[12] K. O'shea and R. Nash, "An introduction to convolutional neural networks," 2015, *arXiv:1511.08458*.

[13] W. Kang and J. J. McAuley, "Self-attentive sequential recommendation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2018, pp. 197–206.

[14] Z. He, H. Zhao, Z. Lin, Z. Wang, A. Kale, and J. J. McAuley, "Locker: Locally constrained self-attentive sequential recommendation," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 3088–3092.

[15] H. Li, Y. Yang, J. Zeng, M. Gao, and J. Wen, "Multi-scale transformers with dual attention and adaptive masking for sequential recommendation," *Inf. Process. Manag.*, vol. 63, no. 1, 2026, Art. no. 104318.

[16] Z. Yue, Y. Wang, Z. He, H. Zeng, J. J. McAuley, and D. Wang, "Linear recurrent units for sequential recommendation," in *Proc. 17th ACM Int. Conf. Web Search Data Mining (WSDM)*, 2024, pp. 930–938.

[17] C. Liu, J. Lin, H. Liu, J. Wang, and J. Caverlee, "Behavior-dependent linear recurrent units for efficient sequential recommendation," in *Proc. 33th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2024, pp. 1430–1440.

[18] Z. Liu et al., "SIGMA: Selective gated mamba for sequential recommendation," in *Proc. 39th AAAI Conf. Artif. Intell. (AAAI)*, 2025, pp. 12264–12272.

[19] J. Tang and K. Wang, "Personalized top-n sequential recommendation via convolutional sequence embedding," in *Proc. 11th ACM Int. Conf. Web Search Data Mining (WSDM)*, 2018, pp. 565–573.

[20] X. Zheng et al., "Enhanced self-attention mechanism for long and short term sequential recommendation models," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 8, no. 3, pp. 2457–2466, Jun. 2024.

[21] R. Wang, C. Fang, X. Sui, Y. Ren, and X. Zhang, "BiSGR-Att: A sequence recommendation model based on bidirectional simplified gated recurrent network with linear attention," *Appl. Soft Comput.*, vol. 190, 2026, Art. no. 114567.

[22] J. Chen et al., "Enhancing explainable sequential recommendation with disentangled representations and auxiliary review explanations," *IEEE Trans. Comput. Soc. Syst.*, vol. 13, no. 1, pp. 783–798, Feb. 2026.

[23] Z. Yang, W. Hu, T. Zhang, Y. Cheng, and X. Wang, "Debiased sequential recommendation by separating long-term and short-term interests," *IEEE Trans. Comput. Soc. Syst.*, vol. 12, no. 6, pp. 4620–4633, Dec. 2025.

[24] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5574–5584.

[25] K. Zhou, H. Yu, W. X. Zhao, and J. Wen, "Filter-enhanced MLP is all you need for sequential recommendation," in *Proc. 31st Int. Conf. World Wide Web (WWW)*, 2022, pp. 2388–2399.

[26] H. Xu, H. Yuan, G. Liu, J. Fang, L. Zhao, and P. Zhao, "Wavelet enhanced adaptive frequency filter for sequential recommendation," in *Proc. 40th AAAI Conf. Artif. Intell. (AAAI)*, 2026, pp. 16058–16065.

[27] Y. Koren, R. M. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[28] S. Kabbur, X. Ning, and G. Karypis, "FISM: Factored item similarity models for top-N recommender systems," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (SIGKDD)*, 2013, pp. 659–667.
[29] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[30] A. Yan, S. Cheng, W. Kang, M. Wan, and J. J. McAuley, "CosRec: 2D convolutional neural networks for sequential recommendation," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2019, pp. 2173–2176.
[31] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS),* 2017, pp. 5998–6008.
[32] J. Li, P. Ren, Z. Chen, Z. Ren, T. Lian, and J. Ma, "Neural attentive session-based recommendation," in *Proc. 26th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2017, pp. 1419–1428.
[33] F. Sun et al., "BERT4Rec: Sequential recommendation with bidirectional encoder representation from Transformer," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2019, pp. 1441–1450.
[34] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. 10th Int. Conf. Learn. Representations (ICLR),* 2022.
[35] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, Ç. Gülçehre, R. Pascanu, and S. De, "Resurrecting recurrent neural networks for long sequences," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, vol. 202, pp. 26670–26698.
[36] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré, "Hungry Hungry Hippos: Towards language modeling with state space models," in *Proc. 11st Int. Conf. Learn. Represent. (ICLR)*, 2023.
[37] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS),* 2022.
[38] Z. Fan et al., "Sequential recommendation via stochastic self-attention," in *Proc. 31st Int. Conf. World Wide Web (WWW)*, 2022, pp. 2036–2047.
[39] S. H. Lim, N. B. Erichson, L. Hodgkinson, and M. W. Mahoney, "Noisy recurrent neural networks," in *Adv. Neural Inform. Process. Syst. 34 (NeurIPS)*, pp. 5124–5137, 2021.
[40] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods," *Mach. Learn*, vol. 110, pp. 457–506, 2021.
[41] Z. Zheng and Y. Yang, "Rectifying pseudo label learning via uncertainty estimation for domain adaptive semantic segmentation," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1106–1120, Mar. 2021.
[42] M. Abdar et al., "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," *Inf. Fusion*, vol. 76, pp. 243–297, 2021.
[43] Z. Fan, Z. Liu, S. Wang, L. Zheng, and P. S. Yu, "Modeling sequences as distributions with uncertainty for sequential recommendation," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, Dec. 2021, pp. 3019–3023.
[44] K. P. Neupane, E. Zheng, and Q. Yu, "MetaEDL: Meta evidential learning for uncertainty-aware cold-start recommendations," in *Proc. IEEE Int. Conf. Data Mining (ICDM),* 2021, pp. 1258–1263.
[45] C. Wang, F. Feng, Y. Zhang, Q. Wang, X. Hu, and X. He, "Rethinking missing data: Aleatoric uncertainty-aware recommendation," *IEEE Trans. Big Data*, vol. 9, no. 6, pp. 1607–1619, Dec. 2023.
[46] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 7482–7491.
[47] X. Wang and S. Kadioglu, "Bayesian deep learning based exploration-exploitation for personalized recommendations," in *Proc. 31st Int. Conf. Tools Artif. Intell. (ICTAI),* 2019, pp. 1715–1719.
[48] Z. Xu, P. Wei, W. Zhang, S. Liu, L. Wang, and B. Zheng, "UKD: Debiasing conversion rate estimation via uncertainty-regularized knowledge distillation," in *Proc. 31st Int. Conf. World Wide Web (WWW)*, 2022, pp. 2078–2087.
[49] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
[50] L. J. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
[51] S. De et al., "Griffin: Mixing gated linear recurrences with local attention for efficient language models," 2024, *arXiv:2402.19427*.
[52] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GeLUs)," 2016, *arXiv:1606.08415*.
[53] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
[54] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 19:1–19:19, 2016.
[55] R. He and J. J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering," in *Proc. 15th Int. Conf. World Wide Web (WWW)*, 2016, pp. 507–517.
[56] J. J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in *Proc. 38th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr. (SIGIR)*, 2015, pp. 43–52.
[57] Z. Yue, Z. He, H. Zeng, and J. J. McAuley, "Black-box attacks on sequential recommenders via data-free model extraction," in *Proc. 15th ACM Conf. Recommender Syst. (RecSys)*, 2021, pp. 44–54.
[58] Z. Yue, H. Zeng, Z. Kou, L. Shang, and D. Wang, "Defending substitution-based profile pollution attacks on sequential recommenders," in *Proc. 16th ACM Conf. Recommender Syst. (RecSys)*, 2022, pp. 59–70.
[59] Y. Shin, J. Choi, H. Wi, and N. Park, "An attentive inductive bias for sequential recommendation beyond the self-attention," in *Proc. 38th AAAI Conf. Artif. Intell (AAAI)*, 2024, pp. 8984–8992.
[60] C. Zhang and X. Hong, "Challenging the long tail recommendation on heterogeneous information network," in *Proc. IEEE Int. Conf. Data Mining Workshops (ICDM)*, 2021, pp. 94–101.



Jian Yang received the Ph.D. degree in pattern recognition and intelligent system from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2006.

He is an Associate Professor with the School of Information Science and Technology, Beijing University of Technology, Beijing. His research interests include data mining, machine learning, web intelligence, and affective computing.



Tengfei Bi received the bachelor's degree in electrical engineering and automation from Zhengzhou University, Zhengzhou, China, in 2022. He is currently working toward the master's degree in control science and engineering with the School of Information Science and Technology, Beijing University of Technology, Beijing, China.

His research interests include recommender systems and data mining.



Jiajin Huang received the Ph.D. degree in computer science and technology from Beijing University of Technology, Beijing, China, in 2007.

He is currently an Assitant Professor with the School of Information Science and Technology, Beijing University of Technology. His research interests include data mining, machine learning, and web intelligence.



Mi Li (Member, IEEE) received the Ph.D. degree in computer science and technology from Beijing University of Technology, Beijing, China, in 2012.

She is a Professor with the School of Information Science and Technology, Beijing University of Technology. Her research interests include affective computing, machine learning, computational psychophysiology, and cognitive neuroscience.

Authorized licensed use limited to: Khon Kaen University provided by UniNet. Downloaded on July 08,2026 at 08:57:47 UTC from IEEE Xplore. Restrictions apply.